

Parcial 2

Oscar Vargas Pabon

Codigo en la carpeta /midterm2/ del repositorio en

https://github.com/osvarp/paralelComputing_week3

Video en

https://drive.google.com/file/d/15B0McJjEhq0CX6xVeX6NB3xDWjloRQ-O/view?usp=drive_link

Actividad 1

Se implemento bien

```
japeto@f0a9d266733c:~/app$ ./act1.out
LAIB#KGCDHENMJF
L A I B
# K G C
D O H E
N M J F

_____Posible movements ->
UP
DOWN
RIGHT

_____Insert a movement:
RIGHT
L A I B
K # G C
D O H E
N M J F
```

Actividad 2

Durante todo el código nombro como A*1 el de la heurística de los elementos ‘bien ubicados’, mientras que A*2 el de la distancia manhattan.

```
japeto@f0a9d266733c:~/app$ ./act2.out
EC#DBAJHIGFLMNKO
bfs 14
A*1 14
A*2 14
```

TAREA7: Notar que desde el comienzo funciono con tableros de tamaño dinámico. Mi clase Table antes requiere de las dimensiones para funcionar (ni si quiera debe ser una matriz cuadrada).

Actividad 3

Mi estrategia de paralelización fue paralelizar el ciclo en donde exploro los distintos estados que puedo alcanzar desde mi frontera.

```
japeto@fe9443ea8cbb:~/app$ ./act3.out
cores 8

EC#DBAJHIGFLMNKO
bfs 14
A*1 14
A*2 14
```

Actividad 4

Midiendo el speedup con tableros pseudo-aleatorios de tamaño 2x2. Cada uno fue testado con 2 tableros.

```
Working with seed <1541050579> and size <2X2>.

Answer format [finished amount] <avg.speedup core1> ... <avg.speedup corei>
Alg order:
1. BFS | 2. A*_1 | 3. A*_2 | <- SEQ
4. BFS | 5. A*_1 | 6. A*_2 | <- OMP
[2] 0.316741 0.62493 0.605821
[2] 0.858203 0.561286 0.536705
[2] 0.506803 0.616989 0.511898
japeto@d7192b06e63d:~/app$ ./act4.out 2 2
Working with seed <-1868907580> and size <2X2>.

Answer format [finished amount] <avg.speedup core1> ... <avg.speedup corei>
Alg order:
1. BFS | 2. A*_1 | 3. A*_2 | <- SEQ
4. BFS | 5. A*_1 | 6. A*_2 | <- OMP
[2] 0.24336 0.00099412 0.000553722
[2] 0.00848729 0.00473616 0.000364705
[2] 0.00811331 0.00430022 0.000267312
japeto@d7192b06e63d:~/app$ ./act4.out 2 2
Working with seed <579175167> and size <2X2>.

Answer format [finished amount] <avg.speedup core1> ... <avg.speedup corei>
Alg order:
1. BFS | 2. A*_1 | 3. A*_2 | <- SEQ
4. BFS | 5. A*_1 | 6. A*_2 | <- OMP
[2] 0.0732459 0.000881031 0.000507547
[2] 0.14155 0.000498284 0.000250268
[2] 0.00244371 0.00111246 0.000274875
```

El [2] significa que todos los pudo terminar (de 4X4 en adelante no todos son posibles por la memoria que requieren). Los siguientes 3 numeros son speedup promedio de BFS, A*_1 y A*_2. La primera línea se hizo con 2 hilos, la segunda con 4 y la tercera con 8.

Similarmente con tableros de 3x3

```
japeto@5aa93b6c2fa6:~/app$ ./act4.out 2 3
Working with seed <-349964809> and size <3X3>.

Answer format [finished amount] <avg.speedup core1> ... <avg.speedup corei>
Alg order:
1. BFS | 2. A*_1 | 3. A*_2 | <- SEQ
4. BFS | 5. A*_1 | 6. A*_2 | <- OMP
[2] 0.863686 0.542896 0.31839
[2] 0.742654 0.535199 0.0969689
[2] 0.866382 0.504019 0.0174793
japeto@5aa93b6c2fa6:~/app$ ./act4.out 2 3
Working with seed <-1324153938> and size <3X3>.

Answer format [finished amount] <avg.speedup core1> ... <avg.speedup corei>
Alg order:
1. BFS | 2. A*_1 | 3. A*_2 | <- SEQ
4. BFS | 5. A*_1 | 6. A*_2 | <- OMP
[2] 0.826774 0.515408 0.307509
[2] 0.681992 0.267468 0.110533
[2] 0.813655 0.171487 0.0130728
```

Por ahora parece ocurrir que el que menos empeoro fue el basado en el bfs. Tambien da la impresión de que los algoritmos funcionan mejor con 4 o 2 hilos. Esto ultimo puede ocurrir porque hemos estado viendo casos pequeños.

```
japeto@4c45e42be104:~/app$ ./act4.out 2 3
Working with seed <1956827097> and size <3X3>.

Answer format [finished amount] <avg.speedup core1> ... <avg.speedup corei>
Alg order:
1. BFS | 2. A*_1 | 3. A*_2 | <- SEQ
4. BFS | 5. A*_1 | 6. A*_2 | <- OMP
[2] 0.601269 0.487952 0.347222
[2] 0.772556 0.67985 0.277922
[2] 0.720989 0.58269 0.177672
japeto@4c45e42be104:~/app$ ./act4.out 10 3
Working with seed <-1389447203> and size <3X3>.

Answer format [finished amount] <avg.speedup core1> ... <avg.speedup corei>
Alg order:
1. BFS | 2. A*_1 | 3. A*_2 | <- SEQ
4. BFS | 5. A*_1 | 6. A*_2 | <- OMP
[10] 0.770722 0.625971 0.356586
[10] 0.764851 0.576038 0.217708
[10] 0.784143 0.581311 0.134498
```

Este ultimo caso fue ejecutado con 10 tableros pseudo-aleatorios. Este parece describir una tendencia a mejorar conforme aumentan los hilos para el de BFS, mientras que empeora con las otras estrategias.